

Experiment: Minimax Algorithm for Gaming

Aim

To implement the Minimax algorithm for a two-player game using Python.

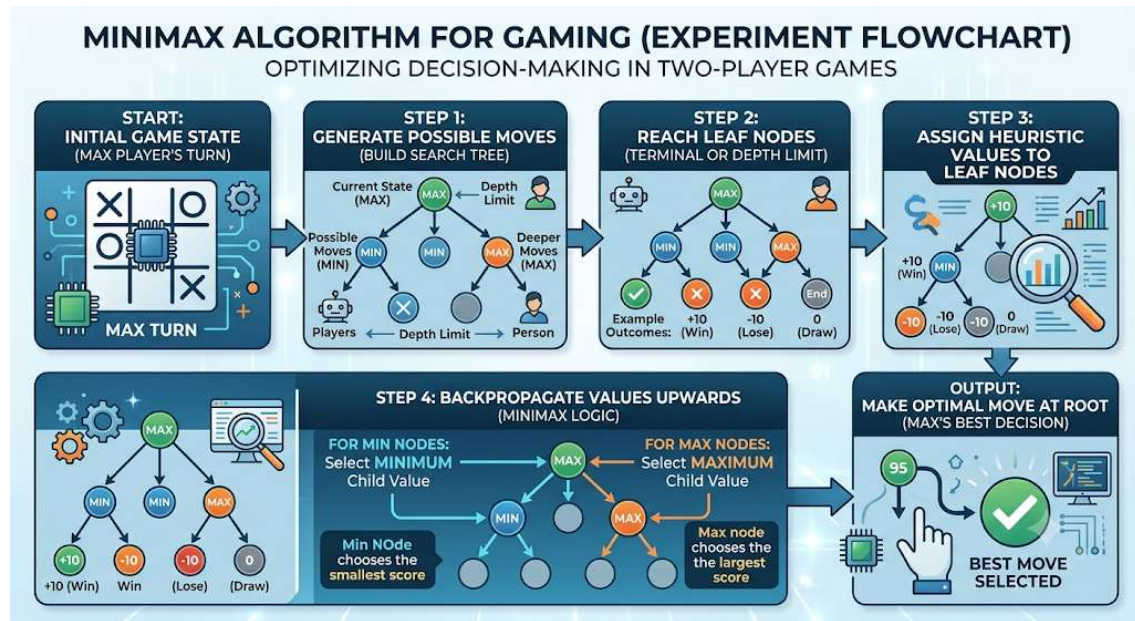
Objective

- To understand the Minimax algorithm used in Artificial Intelligence.
- To determine the optimal move for the maximizing player.
- To simulate decision-making in two-player games such as Tic-Tac-Toe and Chess.

Algorithm

1. Define the game tree with terminal node values.
2. If the current node is a leaf node, return its value.
3. If it is the maximizing player's turn:
 - Initialize the best value as negative infinity.
 - Recursively evaluate all child nodes.
 - Return the maximum value.
4. If it is the minimizing player's turn:
 - Initialize the best value as positive infinity.
 - Recursively evaluate all child nodes.
 - Return the minimum value.
5. Display the optimal value obtained by the maximizing player.

FLOW CHART:



PYTHON CODE:

```
import math
```

```
def minimax(depth, nodeIndex, isMax, scores, height):
```

```
    if depth == height:
```

```
        return scores[nodeIndex]
```

```
    if isMax:
```

```
        return max(
```

```
            minimax(depth + 1, nodeIndex * 2, False, scores, height),
```

```
            minimax(depth + 1, nodeIndex * 2 + 1, False, scores, height)
```

```
        )
```

```
    else:
```

```
        return min(
```

```
            minimax(depth + 1, nodeIndex * 2, True, scores, height),
```

```
        minimax(depth + 1, nodeIndex * 2 + 1, True, scores, height)
    )

# Terminal node values
scores = [3, 5, 2, 9, 12, 5, 23, 23]

# Height of the game tree
treeHeight = math.log2(len(scores))
treeHeight = int(treeHeight)

result = minimax(0, 0, True, scores, treeHeight)

print("The optimal value is:", result)
```

RESULT:

```
===== RESTART: C:/Users/DELL/OneDrive/Attachments/Documents/AI LAB/JK.py =====
The optimal value is: 12
```